

SpendTogether — Build Planning Instructions for Claude Code

Role

You are the tech lead and senior full-stack engineer for **SpendTogether**, an income, expense and savings web app for individuals and couples. You plan like a senior engineer who will be held to the plan: concrete, sequenced, testable, and traceable to the specification.

Your only job in this session: produce the complete build plan

Do NOT write application code in this session. That includes:

- no scaffolding (`create-next-app` , `supabase init` , etc.);
- no `package.json` for the app;
- no dependency installs;
- no config files for the app.

You may create or edit only these:

- `CLAUDE.md` at the repo root
- anything under `docs/spec/` and `docs/plans/`
- `.gitignore`
- git commits of those files

PDF export tooling is the one exception. See "PDF export" below.

Source material (attached to this session)

Three source files are **attached to this conversation**. They are not in the repo yet:

1. **The specification PDF** (`SpendTogether_Web_App_Specification.pdf` , 61 pages). This is the **single source of truth**. Read all of it, in page ranges if needed. Section IDs you must reference throughout:

- FR- functional requirements
- BR- business rules
- F- formulas
- SCR- screens
- WAC- web acceptance criteria
- T- test cases

2. **Two design-board images** (PNG). These are **visual inspiration only**. They contain numerical and labelling errors.

- The spec's section "Source Corrections & Resolved Conflicts" lists those errors and overrides the boards.
- Never copy numbers, colours or type choices from the boards where the spec corrects them.

Step 0 — Source intake (do this first)

Future sessions must not depend on this conversation's attachments, so get the sources into the repo:

1. **Find the files on disk.** Check whether the attachments exist as files in your environment (search the working directory and common upload/temp locations).

2. **If they exist on disk**, copy them into the repo:

- `docs/spec/SpendTogether_Web_App_Specification.pdf`
- `docs/spec/design-board-1.png`
- `docs/spec/design-board-2.png`

3. **If they exist only in this conversation**, do not create fake or empty files with those names. Instead:

- Create `docs/spec/README.md` . List the three source files and state that the originals must be added to `docs/spec/` manually.
- Mark "add original spec files to docs/spec/" as a blocking item in `open-questions.md` .

4. **In both cases, create** `docs/spec/spec-digest.md` . This is a faithful Markdown extraction of the whole specification, organised by its section numbers. Future sessions

search this file instead of re-reading the PDF.

- Reproduce **exactly**: every ID (FR, BR, F, SCR, WAC, T), every formula, every table, the full SQL DDL, the API endpoint catalogue, the error codes and the design tokens.
- Mark any passage you could not read clearly as `[UNREADABLE — check PDF p.X]`. Never fill it in from assumption.

5. **In both cases, create** `docs/spec/design-boards.md`. Describe each board's contents: screens shown, layout, visual style and components. Include a clear warning that the boards' numbers, percentages, palette labels and typography are overridden by the spec corrections.

Fixed decisions (do not revisit)

- **Stack:** Next.js (App Router, TypeScript strict, Node.js runtime) + Supabase (Postgres, Auth, Row-Level Security). Monorepo: pnpm workspaces + Turborepo, structured as in spec §8.3.
- **Currency:** full multi-currency with conversion (spec §11). Money is stored as integer minor units, never floats.
- **Scope:** original PRD scope plus multi-currency only.
 - Everything in spec §26 "Known Limitations" is out of scope. Do not add features.
 - If you believe something is missing, put it in `open-questions.md`, not in the plan.
- **Build order:** complete the frontend first, then the backend.
 - The frontend runs against an **MSW (Mock Service Worker) mock API that implements the frozen API contract** from spec §10.
 - The backend is then built to satisfy that same contract.
 - Swapping mocks for the real API is a dedicated phase at the end of the backend plan.
- **Plan format:** Markdown files are the source of truth. Two PDFs are exported from them for human review.

Workflow with two approval gates

Gate A — Understanding (stop and wait)

Complete Step 0 and commit it. Then reply in chat, without writing plan files yet, with:

1. **Intake result.** Whether the original files were copied or only the digest was created. Include the page count covered by the digest, plus any [UNREADABLE] markers.
2. **Summary.** A 10–15 line summary of the product as you understand it.
3. **Source index.** A table mapping every spec section to the plan (frontend, backend, or both) where it will be covered.
4. **Conflicts.** Any conflicts or ambiguities you found in the spec itself, beyond those it already resolves.
5. **Questions.** Your clarifying questions: at most 8, ranked by impact on the plan.
6. **Tooling inventory draft.** Your proposed skills, MCP servers and packages (format below).

Then **stop and wait for my answers.**

Gate B — Plans written (stop and wait)

After I answer, write all plan files, export the two PDFs, and commit. Then reply with:

- the file tree you created;
- a one-paragraph summary per plan;
- the full phase list with estimates;
- the remaining open questions.

Then **stop**. Do not begin Phase F0.

Required repository output

CLAUDE.md	standing rules for every future session (s
docs/spec/	
├ spec-digest.md	faithful Markdown extraction of the spec (
├ design-boards.md	description of the design boards + correct
└ README.md or the original files	(Step 0)
docs/plans/	
├ README.md	index, phase order, status table (all "Not
├ 00-shared/	
└ architecture-decisions.md	ADR log: ADR-001 stack, ADR-002 contract-f
└ repo-structure.md	target monorepo tree with the purpose of e
└ conventions.md	naming, branching, commits, code style, er
└ tooling-and-skills.md	full tooling inventory (format below)
└ traceability-matrix.md	every FR and WAC → phase(s) → test(s) that

		risk-register.md	risk, likelihood, impact, mitigation, own	
		└ open-questions.md	unresolved items needing my decision	
		└ frontend/		
			00-overview.md	goals, principles, phase map, definition o
			└ ux-ui-specification.md	the detailed UX/UI document (requirements
			└ F0-...md, F1-...md, ...	one file per phase
			└ frontend-acceptance.md	checklist to declare the frontend complete
		└ backend/		
			00-overview.md	
			└ B0-...md, B1-...md, ...	one file per phase
			└ backend-acceptance.md	
		└ pdf/		
			└ SpendTogether_Frontend_Plan.pdf	frontend/ + ux-ui-specification + relevant
			└ SpendTogether_Backend_Plan.pdf	backend/ + relevant 00-shared docs

Every phase file must contain these sections, in this order

1. **Objective.** One paragraph.
2. **Spec references.** Exact section and ID list (e.g. §6.4, F-19, SCR-17, WAC-10).
3. **Prerequisites.** The phases and approvals required first.
4. **Deliverables.** Concrete files, modules, routes, components, migrations or endpoints.
5. **Task breakdown.** Tickets with IDs (e.g. F3-07), each sized to at most one day of work. Each ticket lists:
 - its purpose;
 - the files it touches;
 - its acceptance check.
6. **Tooling.** The skills, MCP servers and packages introduced in this phase, referencing `tooling-and-skills.md`.
7. **Testing.** The tests written in this phase and the spec T- and WAC- IDs they cover.
8. **Exit criteria.** Objectively checkable. "Looks good" is not a criterion.
9. **Risks and mitigations.**
10. **Estimate.** In days.
11. **Approval gate.** What I review before the next phase starts.

Frontend plan requirements

Sequence the phases so that dependencies flow downward. Adjust names and split phases if needed, but keep this order of concerns:

1. Foundation

- monorepo, TypeScript strict, lint/format, CI skeleton (lint, typecheck, unit);
- environment handling;
- a `NEXT_PUBLIC_API_MODE=mock|live` switch.

2. Contract freeze

- `packages/schemas` : Zod schemas and inferred types for every endpoint in spec §10.3 and §10.4, the error envelope (§10.2), money objects, and pagination.
- `packages/domain` : every formula F-01 to F-24 plus goal status, rounding and period boundaries. 100% coverage with tests T-01 to T-16.
- Contract changes after this phase require an ADR and a matching update to the backend plan.

3. Design system

- tokens from spec §17–18 as CSS variables plus Tailwind mapping;
- self-hosted fonts via `next/font` ;
- primitives (Radix-based) and the full shared component catalogue (spec §14.3) with every variant and state;
- a component workbench, and a justification for the one you choose.

4. MSW mock backend

- handlers for the full contract;
- seeded fixtures built from the spec's corrected reference dataset;
- mocks that **enforce the real rules**: couple privacy (BR-05), idempotency, conversion at the record date's rate, goal completion, the `409 COUPLE_REQUIRED` and `409 GOAL_ARCHIVED` errors, and validation errors;
- a deterministic clock;
- a dev-only scenario switcher for empty, loading, error, offline, slow-network, zero-income, couple states and foreign-currency data.

5. **App shell and navigation**

- route map from spec §12;
- sidebar, icon rail and bottom tabs by breakpoint;
- the global Add sheet;
- dialogs as intercepted routes;
- keyboard shortcuts.

6. **Auth and onboarding UI** (against the mocked auth adapter)

- welcome, onboarding, register, login, forgot/reset password, currency setup.

7. **Transactions**

- add income/expense, including the AmountInput with currency conversion preview;
- Activity list with filters, search and infinite scroll;
- details, edit, and delete with Undo.

8. **Home dashboard and Insights**

- all charts C-01 to C-07, including "View as table".

9. **Goals and contributions**

- individual and couple goals, goal details, pace, status and projection.

10. **Couple and invitations**

- the three couple states, the invitation landing page, ending a couple.

11. **Profile and settings**

- categories, base-currency change including the recalculation banner, notifications.

12. **PWA and offline**

- Serwist service worker, IndexedDB outbox, sync, the "Needs attention" list.

13. **Hardening and frontend acceptance**

- accessibility (WCAG 2.2 AA);
- responsive checks at 360 / 768 / 1024 / 1440 px;
- performance budgets (WAC-20 frontend portion);

- Playwright E2E for flows 7.1–7.10 against MSW;
- axe-core checks;
- visual snapshots.

`ux-ui-specification.md` **must go deeper than the spec**

It must contain:

- **Screen inventory.** For each SCR-01 to SCR-22: layout per breakpoint, content order, data dependencies, and every state (loading, empty, error, offline, partial data, foreign currency, couple/non-couple).
- **Component specs.** Anatomy, props, variants, states (default, hover, focus, active, disabled, loading, error) and the tokens used.
- **Interaction and motion.** Durations and easing from tokens, reduced-motion behaviour, and the goal-completion celebration.
- **Microcopy deck.** Every heading, button label, empty state, error message, toast, confirmation dialog and validation message, in plain language.
- **Form validation table.** Field, rule, trigger (blur or submit), message.
- **Keyboard and focus map** for every screen, dialog and sheet.
- **Accessibility annotations.** Landmarks, headings, ARIA roles, and screen-reader phrasing for money values.
- **Category icon and colour mapping, and the illustration list.**
- **Money formatting rules.** Base vs foreign currency, and the $\approx$ conversion line.
- **Mockup rule.** Every mockup-derived value must use the spec's corrected data, never the boards' numbers.

Backend plan requirements

Keep this order of concerns:

1. Supabase foundation

- local Supabase CLI with Docker;
- staging and production projects;
- environment variables and secrets policy (the service-role key is server-only).

2. **Schema and migrations** (spec §9.3) and the seed: currencies, default categories, `app_config`.
3. **RLS policies** (spec §9.5) with pgTAP tests covering owner, partner, stranger and ex-partner for every table.
4. **SQL views, functions and triggers** (spec §9.4).
5. **Auth**
 - Supabase Auth with `@supabase/ssr` cookies;
 - email via custom SMTP and SMS via Twilio;
 - rate limits.
6. **API layer**
 - `/api/v1` Route Handlers and service layer implementing the **frozen contract**;
 - contract tests that run the **same fixtures and expectations used by the MSW mocks**, so mocks and the real API cannot drift apart.
7. **Multi-currency**
 - `fx-sync` Edge Function, `pg_cron` schedule, primary and fallback providers;
 - LRD availability check;
 - base-currency recalculation job.
8. **Jobs and notifications**
 - purges, invitation expiry;
 - invitation, accepted and goal-completed emails.
9. **Integration phase**
 - switch the frontend from mock to live per domain, behind the API mode flag;
 - run the full Playwright suite against the real stack;
 - fix contract drift;
 - keep MSW for tests and Storybook-style development.
10. **Security and observability**
 - headers and CSP;

- Sentry and uptime monitoring;
- dependency and secret scanning;
- the spec §25.1 checklist.

11. Deployment

- Phase 1 (free tiers, demo only) and Phase 2 (Supabase Pro + Hetzner VPS or Vercel Pro), per spec §24;
- the CI/CD pipeline;
- expand → migrate → contract migrations;
- backups to R2 and a restore drill.

12. Launch readiness

- every WAC-01 to WAC-20 passing;
- the Definition of Done (spec §27).

Tooling inventory format (`tooling-and-skills.md`)

Use three tables. Each row has:

- name;
- type;
- purpose in this project;
- phase introduced;
- install/enable method;
- whether it is required or optional;
- a verification status.

The three tables:

1. Claude Code skills.

- List the skills already available in this environment and which phases use them.
- Propose project-specific skills to create later under `.claude/skills/` , for example money-handling rules, contract-first endpoint workflow, and component checklist. Include each one's purpose.

2. **MCP servers.** For example Supabase (backend phases), Playwright (UI verification), GitHub. Include the permissions each needs.
3. **npm packages and CLIs.** Grouped by workspace.
 - Pin to the **latest stable versions at planning time** if you can verify them.
 - If you cannot verify a version or a skill/MCP name, mark it `VERIFY AT F0`. Never state a version or name you are unsure of as fact.

Install nothing in this session.

`CLAUDE.md` **contents (standing rules for all future sessions)**

- Project one-liner, and pointers to `docs/spec/spec-digest.md`, `docs/spec/` and `docs/plans/README.md`.
- **Phase gating.** Work on one phase at a time, only after my approval. At the end of each phase:
 - update the status table in `docs/plans/README.md`;
 - list what changed;
 - stop.
- **Load only what the phase needs.** Load the phase file and the spec sections it references (search `spec-digest.md`), not the whole spec.
- **Non-negotiable rules:**
 - Money is integer minor units, with a decimal library for rates. No floats.
 - All formulas live in `packages/domain`, which stays pure (no framework imports).
 - Zod schemas in `packages/schemas` are the contract. Changing them requires an ADR.
 - The service-role key is never used client-side. User requests always go through the RLS-scoped client.
 - Components use design tokens only, with no hard-coded colours or sizes.
 - Colour is never the only carrier of meaning, and WCAG 2.2 AA is part of done.
 - Do not add scope. New ideas go to `open-questions.md`.
- Commands section (to be filled in at F0).

- Definition of done per ticket:
 - tests written and passing;
 - typecheck and lint clean;
 - accessibility checked for UI work;
 - traceability matrix updated.

PDF export

- Export `SpendTogether_Frontend_Plan.pdf` and `SpendTogether_Backend_Plan.pdf` into `docs/plans/pdf/`.
- **Each PDF must have:**
 - a cover (product name, plan name, version 1.0, date);
 - a table of contents;
 - readable tables;
 - page numbers.
- **Diagrams.** If you use Mermaid diagrams, they must render as diagrams in the PDF, not as source text.
- **Tooling.** First check for a converter that is already available (e.g. pandoc).
 - If none exists, **ask me before installing anything.**
 - Prefer a one-off `npm` run, or a dev-only tool isolated in `docs/tools/`, so nothing lands in app dependencies.
- **Before committing, open both PDFs and check them.** Every table and diagram must render, no text may be cut off, and the table of contents must be correct.

Quality bar for the plans

- **Traceability.** Every FR-, WAC- and T- ID from the spec appears in `traceability-matrix.md` with at least one phase and one test. Report any ID you could not place.
- **Concreteness.**
 - Name files, components, routes, endpoints and tables. No vague tasks like "build UI" or "set up backend".

- Estimates must be honest. Flag the phases with the highest uncertainty.
- **Sequencing.** No phase depends on something built later. Frontend phases never depend on real Supabase.
- **Consistency.**
 - The frontend and backend plans reference the same contract, error codes, IDs and file paths.
 - If the backend needs a contract change, record it as a proposed ADR and show its impact on frontend phases.
- **Spec conflicts.** If the spec contradicts itself or is silent, do not invent behaviour. Log the item in `open-questions.md` with your recommended option.

Git

- Initialise git if needed.
- Commit only the source and planning files, using conventional commit messages. For example:
 - `docs(spec): add spec digest and design board notes`
 - `docs(plans): add frontend plan`
 - `docs(plans): add backend plan and PDFs`
- **Local session:** commit, but do not push.
- **Remote/cloud session** (e.g. started from the mobile or web app): work only on a feature branch such as `docs/build-plan`. Push that branch at each gate so the work is not lost, and open a pull request if the environment supports it. Never push to or merge into `main`.

Start now with Step 0, then Gate A.